

Deep Learning-Based Spectrum Sensing under Partial Observations

Bachelor of Technology

By

Chetan Prakash Dudi (2022CSB1074)

Manish Anwla (2022CSB1090)



Department of Computer Science & Engineering

Indian Institute of Technology Ropar

Rupnagar 140001, India

May 2026

Abstract

Wireless spectrum is scarce and heavily utilized. Cognitive Radio Networks (CRNs) allow Secondary Users (SUs) to access spectrum opportunistically when Primary Users (PUs) are absent. Reliable spectrum sensing is difficult at low Signal-to-Noise Ratio (SNR) and under *partial observations*, where each SU monitors only part of the wideband spectrum. Classical methods (energy detection, cyclostationary detection) degrade under noise uncertainty. Deep learning performs well on centralized full-band data but often fails when observations are heterogeneous and incomplete. This report consolidates our seventh-semester work: (i) a GNU Radio + ADALM-Pluto SDR chain for PU transmission and SU reception with logging of Power Spectral Density (PSD), SNR, and PU presence; (ii) a PyTorch framework for recurrent spectrum sensing models (Vanilla RNN, LSTM, Bi-LSTM, GRU, enhanced gated LSTM) on synthetic PSD sequences; (iii) benchmarking against collaborative CNN ideas from recent literature; and (iv) quantitative results on convergence, probability of detection at fixed false-alarm rate, and ROC behaviour across SNR regimes.

Acknowledgements

We thank our guides Dr. Sudeepta Mishra and Anjani Gupta for their sustained mentorship on cognitive radio systems, software-defined radio experimentation, and deep learning for wireless sensing.

Honor Code

We certify that we have properly cited any material taken from other sources and have obtained permission for any copyrighted material included in this report. We take full responsibility for any code submitted as part of this project and the contents of this report.

Chetan Prakash Dudi (2022CSB1074)

Manish Anwla (2022CSB1090)

Certificate

It is certified that the B. Tech. project “Deep Learning-Based Spectrum Sensing under Partial Observations with Pluto SDR Validation” has been carried out by Chetan Prakash Dudi (2022CSB1074) and Manish Anwla (2022CSB1090) under our supervision. This report is submitted towards partial fulfillment of the B. Tech. project requirements.

Dr. Sudeepta Mishra & Anjani Gupta
Project Supervisors
Department of Computer Science & Engineering
Indian Institute of Technology Ropar
Rupnagar—140001

Contents

Contents	v
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Motivation	1
1.2 Problem statement	1
1.3 Objectives	1
1.4 Scope	2
2 Background and Related Work	3
2.1 Cognitive radio sensing	3
2.2 Collaborative learning under partial observations (literature summary)	3
2.3 Why recurrence	4
3 Pluto SDR and GNU Radio Experimental Path	5
3.1 Complex baseband and I/Q representation	5
3.2 Toolchain	5
3.3 Primary User transmitter (PU_tx.grc)	5
3.4 Secondary User receiver (SU_rx.grc)	6
3.5 Custom psd_logger block	6
3.6 Modulation impact	6
3.7 Logged CSV schema (hardware captures)	7

4	Synthetic Dataset and Channel Models	9
4.1	From measured PSD libraries to multi-user simulation	9
4.1.1	Simulating Single PU-SU to Multiple PUs and SUs	9
4.1.2	Dataset Generation Procedure	11
4.1.3	Dataset Generation Algorithm	12
4.2	Frequency drift	13
4.3	Path loss and shadowing	13
4.4	AWGN calibration	13
5	Deep Learning Methodology	14
5.1	Tensor geometry and labels	14
5.2	Preprocessing	14
5.3	Architectures	14
5.4	Training protocol	15
6	Results and Discussion	16
6.1	Improvement study: Bi-LSTM and training dynamics	16
6.2	Enhanced model with logits, AdamW, and scheduling	16
6.3	ROC AUC trends	17
6.4	Case studies	18
7	Conclusions and Future Work	19
	Bibliography	23

List of Figures

3.1	GNU Radio PU transmitter (<code>PU_tx.grc</code>)	6
3.2	GNU Radio SU receiver with PSD logging (<code>SU_rx.grc</code>)	7
4.1	Spatial placement of multiple Primary Users (PUs) and Secondary Users (SUs) in the synthetic simulator.	11
6.1	P_d versus SNR at fixed $P_{fa} = 5\%$ (replace with your exported matplotlib figure)	17
6.2	ROC curves across SNR (replace with your run artifacts)	17

List of Tables

3.1	Modulation characteristics (qualitative summary from project slides).	7
3.2	Sample rows from measured <code>psd_log_BPSK.csv</code> (truncated precision).	8

Chapter 1

Introduction

1.1 Motivation

Primary Users (PUs) are licensed incumbents with priority access; Secondary Users (SUs) must sense the spectrum and transmit only when harmful interference can be avoided. Hardware limits and spatial variability imply that each SU often sees only a *slice* of the spectrum at a time, motivating collaborative and learning-based sensing under partial observations.

1.2 Problem statement

Efficient spectrum utilization requires accurate PU detection. Traditional sensing struggles at low SNR and under noise uncertainty. Deep models such as CNNs excel on full-band centralized tensors but can underperform when each SU observes a different subset of sub-channels. Temporal ON/OFF behaviour of PUs further motivates recurrent models.

1.3 Objectives

- Implement deep learning detectors (RNN, LSTM, Bi-LSTM, GRU, enhanced gated LSTM) for multi-band occupancy from PSD features.

-
- Preprocess synthetic PSD datasets at SNR levels -16 dB, -10 dB, and -4 dB; evaluate ROC curves, P_d versus SNR, and convergence.
 - Compare recurrent architectures with collaborative CNN-style approaches described in the literature (spatial decoupling, federated aggregation).
 - Validate the sensing pipeline with real over-the-air captures using PlutoSDR and GNU Radio Companion flowgraphs, exporting CSV and PyTorch tensor logs.

1.4 Scope

We developed a PyTorch training and evaluation codebase, a synthetic PSD generator with channel impairments, and Pluto-based PU/SU flowgraphs. The mid-semester document emphasized model theory and implementation; the end-semester presentation integrated the SDR measurement path and dataset formats.

Chapter 2

Background and Related Work

2.1 Cognitive radio sensing

Classical energy detectors are simple but sensitive to noise power uncertainty. Cyclostationary detectors exploit periodic statistics but incur higher complexity. Learning-based detectors map spectral features directly to occupancy probabilities and can fuse heterogeneous inputs when properly architected.

2.2 Collaborative learning under partial observations (literature summary)

Recent work on collaborative spectrum sensing under partial observations proposes: decoupled deep networks split across bands; multi-task learning so each sub-network handles its observed slice; federated-style aggregation where shallow layers are averaged globally and deep layers are averaged among SUs that share the same band visibility; and privacy preservation by exchanging model updates instead of raw PSD snapshots. These ideas improve P_d when each SU sees only consecutive sub-channels.

2.3 Why recurrence

PU activity exhibits temporal structure. Convolutional front-ends emphasize local spatial patterns in PSD tensors; LSTM/GRU stacks model sequential structure across bands or time. Bi-directional recurrence is natural when energy leaks symmetrically to adjacent bands.

Chapter 3

Pluto SDR and GNU Radio Experimental Path

3.1 Complex baseband and I/Q representation

Any passband signal can be represented in complex baseband as $x(t) = I(t) + jQ(t)$, with I the in-phase and Q the quadrature component. GNU Radio processes complex streams; Pluto sources/sinks exchange interleaved I/Q samples.

3.2 Toolchain

NumPy supports vectorized FFTs on complex buffers; PyTorch stores tensors for offline learning; CSV logs capture human-readable traces; Pluto tunes roughly 325 MHz–3.8 GHz with sample rates up to 61.44 MSPS (device configuration dependent).

3.3 Primary User transmitter (PU_tx.grc)

Blocks: File Source $\rightarrow$ Constellation Modulator $\rightarrow$ QT GUI Frequency Sink + PlutoSDR Sink. Representative parameters used in our lab setup: LO 2.4 GHz, sample rate 1.024 MHz, buffer size 32,768 samples. Binary symbol files (e.g., BPSK/QPSK/8PSK/16-QAM) feed the modulator for repeatable experiments.

Transmitter Flowgraph (PU_tx.grc)



Figure 3.1: GNU Radio PU transmitter (PU_tx.grc).

3.4 Secondary User receiver (SU_rx.grc)

Blocks: PlutoSDR Source $\rightarrow$ QT GUI Frequency Sink $\rightarrow$ File Sink $\rightarrow$ custom PSD logger. LO 2.4 GHz, sample rate 1.024 MHz, RF bandwidth 20 MHz (as configured in our flowgraph).

3.5 Custom psd_logger block

The synchronous Python block reads complex samples, estimates PSD in dB, estimates SNR using the mean PSD and a noise floor proxy (10th percentile), and flags PU presence when SNR exceeds a threshold (e.g., > 5 dB). Outputs are written to CSV (timestamped traces) and `.pth` tensors; binned datasets group examples by SNR ranges for training.

3.6 Modulation impact

Table 3.1 summarizes qualitative trade-offs observed in our study: higher-order constellations increase throughput but narrow noise margin, which steepens the

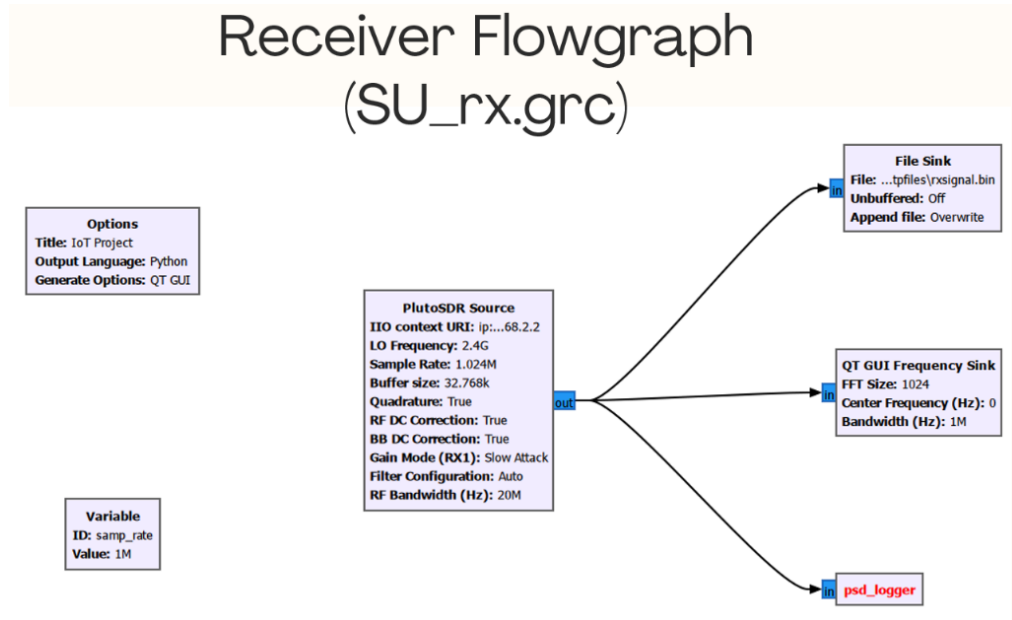


Figure 3.2: GNU Radio SU receiver with PSD logging (SU_rx.grc).

PSD shape and reduces robust sensing at low SNR unless models are retrained or regularized.

Table 3.1: Modulation characteristics (qualitative summary from project slides).

Scheme	Information load	Data rate	Noise robustness
BPSK	1 bit/symbol	Low	High
QPSK	2 bits/symbol	Medium	Medium
8PSK	3 bits/symbol	Medium	Medium–High (implementation dependent)
16-QAM	4 bits/symbol	High	Lower

3.7 Logged CSV schema (hardware captures)

Real-time captures include columns `Timestamp`, `Mean_PSD_dB`, `SNR_dB`, and `PU_Present`.

Table 3.2 shows a few representative rows from `pluto_sdr/pluto_sdr/Results/psd_log_BPSK.csv` as recorded in the repository.

Table 3.2: Sample rows from measured `psd_log_BPSK.csv` (truncated precision).

Timestamp	Mean_PSD_dB	SNR_dB	PU_Present
1762947040.711	-12.09	11.58	1
1762947040.712	-12.26	12.31	1
1762947040.713	-14.31	13.34	1
1762947040.720	-14.16	13.45	1

Chapter 4

Synthetic Dataset and Channel Models

4.1 From measured PSD libraries to multi-user simulation

We combined real PSD libraries captured per modulation (BPSK, QPSK, 8PSK, 16-QAM) with a scalable simulator. A representative configuration discussed in our slides: 10 PUs, 10 SUs, 20 channels, 64 frequency bins per channel, PSD length 192 bins split into left leakage (0–63), main lobe (64–127), and right leakage (128–191).

4.1.1 Simulating Single PU-SU to Multiple PUs and SUs

The original experimental setup consists of a single Primary User (PU) transmitter and a single Secondary User (SU) receiver implemented using PlutoSDR hardware and GNU Radio Companion. Although this setup is sufficient for validating real-time spectrum sensing, large-scale deep learning models require significantly larger datasets involving multiple users, channels, and spectrum occupancy conditions.

To address this limitation, the measured PSD captures obtained from PlutoSDR experiments were used as the foundation for a scalable synthetic dataset

generation framework. Instead of generating artificial PSD patterns from scratch, the simulator reuses real modulation-specific PSD measurements collected from SDR transmissions, thereby preserving realistic spectral characteristics.

The generated simulation environment contains multiple PUs and SUs distributed across a shared wireless spectrum. In our implementation, the system configuration includes:

- Number of Primary Users (PUs): 10
- Number of Secondary Users (SUs): 10
- Number of channels: 20
- Frequency bins per channel: 64
- Total PSD length: 192 bins

Each PSD sample consists of three major regions:

- Left spectral leakage region
- Main signal region
- Right spectral leakage region

To generate realistic wireless observations, several channel impairments are introduced during simulation. These include channel gain variations, path loss attenuation, random shadowing, frequency drift, and additive white Gaussian noise (AWGN). Such impairments emulate practical wireless communication conditions observed in real deployments.

The simulator also models partial spectrum observations, where each SU can observe only a subset of the complete wideband spectrum. This creates the collaborative sensing environment required for studying deep learning-based spectrum sensing under partial observations.

Finally, the generated PSD observations are converted into multi-dimensional tensors and corresponding occupancy labels suitable for training recurrent neural networks such as RNNs, LSTMs, Bi-LSTMs, and GRUs.

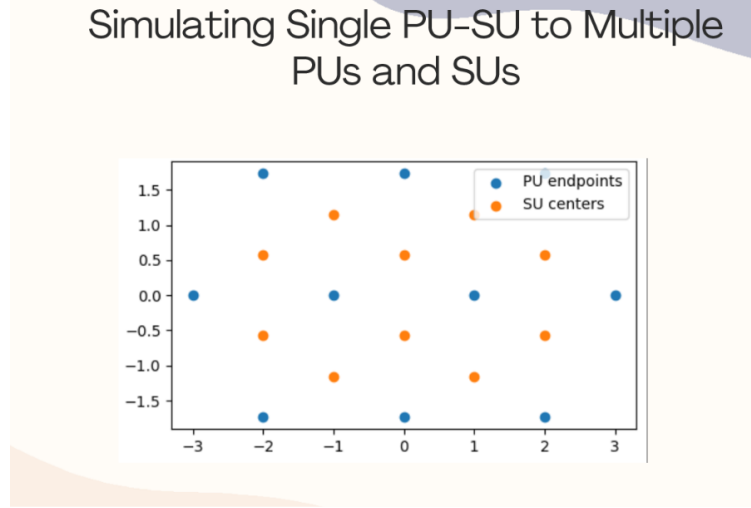


Figure 4.1: Spatial placement of multiple Primary Users (PUs) and Secondary Users (SUs) in the synthetic simulator.

4.1.2 Dataset Generation Procedure

The synthetic dataset generation process begins by randomly selecting PSD samples from the real modulation-specific PSD libraries obtained using PlutoSDR measurements. These PSD samples are then assigned to active Primary Users (PUs) distributed across multiple channels.

For every generated sample, the simulator performs the following steps:

1. Randomly activate a subset of Primary Users.
2. Assign each active PU to a specific spectrum channel.
3. Apply channel gain and distance-based attenuation between PUs and SUs.
4. Introduce shadowing and fading effects to emulate realistic wireless propagation.
5. Apply frequency drift and spectral leakage across neighboring frequency bins.
6. Add Additive White Gaussian Noise (AWGN) according to the target Signal-to-Noise Ratio (SNR).

-
7. Generate PSD observations for all Secondary Users.
 8. Convert the generated PSD observations into tensor representations.
 9. Generate multi-label occupancy vectors corresponding to active spectrum channels.

This procedure enables the generation of large-scale realistic spectrum sensing datasets while preserving the spectral characteristics observed in real SDR measurements.

4.1.3 Dataset Generation Algorithm

[H]

1. Load modulation-specific `.pth` files and build a PSD library.
2. For each class $c \in \{0, \dots, 2^{N_{PU}} - 1\}$:
 - (a) For each sample index i from 1 to desired count:
 - i. Initialize empty input list $\mathcal{J}$ and label vector y of length N_{ch} .
 - ii. Randomly assign a modulation type to each PU and sample a PSD from the corresponding library.
 - iii. For each Secondary User (SU) s :
 - A. Initialize tensor $A \in \mathbb{R}^{1 \times N_w \times N_{ch}}$ to zero.
 - B. For each Primary User (PU) p :
 - C. If bit p of class c is active:
 - D. Compute channel gain $g_{p,s}$.
 - E. Apply frequency drift to the PSD and scale by $g_{p,s}$.
 - F. Compute signal power P_{signal} and select target SNR.
 - G. Derive noise power P_{noise} and add signal plus AWGN into assigned channels.
 - H. Mark occupied channels in label vector y .
 - I. If no PU is active:

-
- J. Add small baseline noise to tensor A .
 - K. Append $|A|$ to $\mathcal{J}$.
 - iv. Append $(\mathcal{J}, y)$ to the dataset.

4.2 Frequency drift

Receivers rarely observe a tone exactly centered due to LO mismatch, PLL wander, Doppler, and filter imperfections. We applied fractional circular shifts $x_{\text{shifted}}[n] \approx x[n - \Delta f]$ to emulate bin-wise drift in the PSD vector.

4.3 Path loss and shadowing

Received power drops with distance and environment. We used a log-distance style gain model with parameters (α, β) and optional log-normal shadowing to capture slow fading around the mean path loss.

4.4 AWGN calibration

Thermal and electronic noise are modeled as AWGN. Noise power is set relative to signal power to realize target SNR: $P_{\text{noise}} = P_{\text{signal}}/\text{SNR}_{\text{linear}}$.

Chapter 5

Deep Learning Methodology

5.1 Tensor geometry and labels

Each training example used a PSD grid of 64×20 frequency bins across bands with a length-20 binary label vector indicating PU occupancy per band. SNR-conditioned datasets enable P_d versus SNR sweeps.

5.2 Preprocessing

Z-score normalization $X' = (X - \mu)/\sigma$ stabilizes optimization. For recurrent models we reshaped inputs to sequences ($T=20$, $F=64$) so each time step corresponds to one band’s spectral slice, enabling inter-band dependency learning.

5.3 Architectures

Vanilla RNN baseline; **LSTM** with two layers and hidden size 128; **Bi-LSTM** concatenates forward/backward states before a dense head; **GRU** for lighter gating; **Enhanced gated Bi-LSTM** with larger width (256), depth (3 layers), layer normalization, dropout, logits head with `BCEWithLogitsLoss`, AdamW, gradient clipping, and `ReduceLROnPlateau` scheduling.

5.4 Training protocol

Optimizer: Adam / AdamW with learning rate 10^{-3} unless noted; batch size 64; 20–50 epochs; binary cross-entropy per band. Evaluation reports ROC/AUC, P_d at fixed $P_{fa} = 5\%$, and training curves.

Chapter 6

Results and Discussion

6.1 Improvement study: Bi-LSTM and training dynamics

Enabling bidirectionality and widening the classifier head improved final “sensing accuracy” derived from training loss. Representative outcomes from our mid-semester report include: Case 1 (full data) at epoch 20, SNR -4 dB improved from $\sim 53.0\%$ to 58.8% (+5.8 points); SNR -10 dB from $\sim 44.8\%$ to 52.0% (+7.2); SNR -16 dB from $\sim 41.0\%$ to 49.4% (+8.4). Case 2 (500-sample subset) exhibited large gains at harsh SNR, e.g. -16 dB from $\sim 69.6\%$ to 92.2% (+22.6 points) with a late-epoch surge after $\sim$ epoch 13.

6.2 Enhanced model with logits, AdamW, and scheduling

With ImprovedLSTMDetector (Bi-LSTM, layer norm, dropout, logits) Case 2 converged to ~ 97.5 – 97.7% across all three SNRs, nearly erasing the SNR gap on the proxy metric. At $P_{fa} = 5\%$, Case 2 P_d moved to $\approx 5.9\%$ (-16 dB), $\approx 10.5\%$ (-10 dB), and $\approx 14.8\%$ (-4 dB), reflecting the difficulty of detection at very low SNR even with strong regularization.

6.3 ROC AUC trends

Case 1 AUC at -4 dB was ~ 0.698 after improvements (small decrease versus an earlier baseline in one ablation), while -10 dB rose to ~ 0.614 ($+0.013$) and -16 dB to ~ 0.531 ($+0.008$). Case 2 AUCs remained near 0.5 – 0.54 , indicating that aggressive fitting on tiny data can inflate training accuracy while leaving discrimination on held-out positives/negatives modest—highlighting the need for calibrated evaluation.

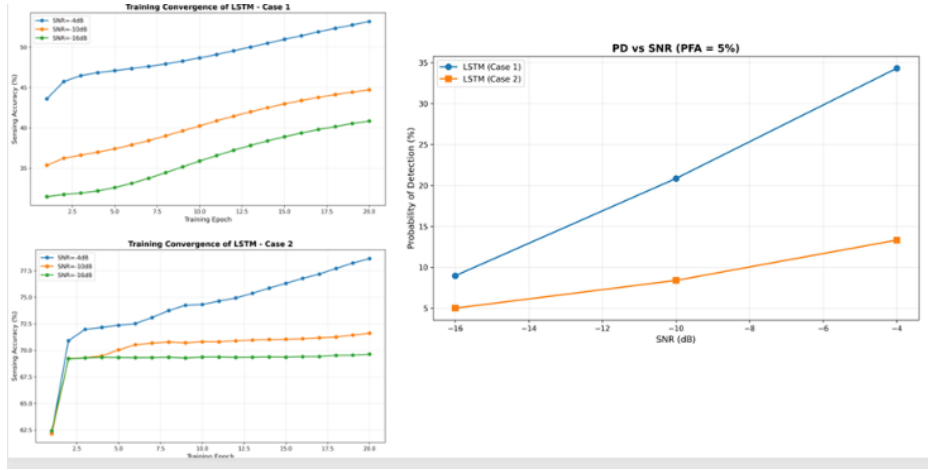


Figure 6.1: P_d versus SNR at fixed $P_{fa} = 5\%$ (replace with your exported matplotlib figure).

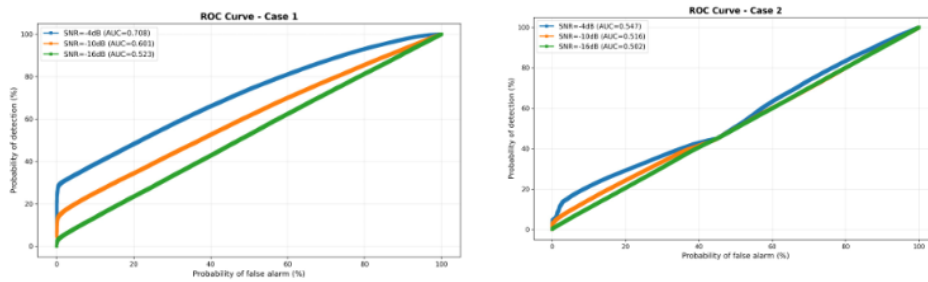


Figure 6.2: ROC curves across SNR (replace with your run artifacts).

6.4 Case studies

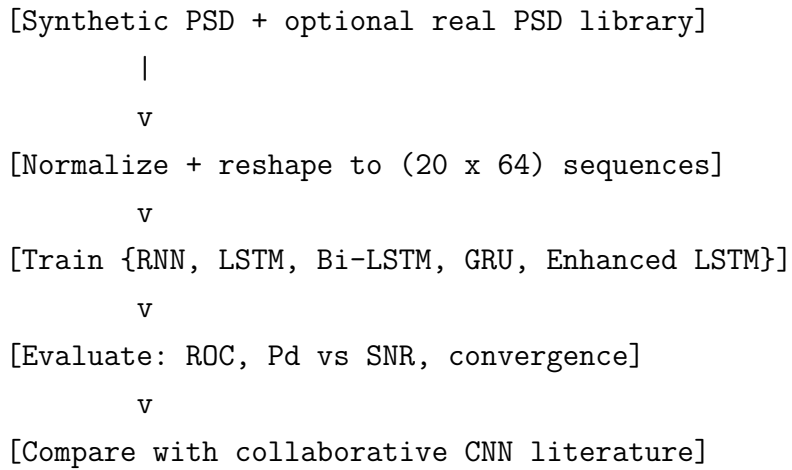
Dynamic modulation mismatch: when PU modulation at test time differs from training, recurrent models may need retraining; collaborative CNNs sometimes generalize better due to shared spectral filters. **Partial band coverage:** centralized RNNs degrade if bands disappear at test time; collaborative decoupled architectures maintain higher P_d via band-specific subnets and targeted aggregation. **Extreme SNR:** Bi-LSTM/enhanced LSTM improve low-SNR P_d versus vanilla RNN, while collaborative CNNs remain strong when spatial sharing is beneficial.

Chapter 7

Conclusions and Future Work

We integrated a Pluto SDR–GNU Radio measurement front-end with a PyTorch spectrum sensing backend, and documented recurrent and collaborative-CNN-style trade-offs under partial observations and harsh SNR. Future directions include hybrid CNN–RNN encoders, transformer-based spectrum transformers, federated training with realistic bandwidth limits, and closing the sim-to-real gap with larger field captures.

Appendix A: Workflow Diagram (Conceptual)



Appendix B: Project Resources and Links (from End-Semester Slide 26)

The following hyperlinks were extracted from Btp1_endsem.pptx (Slide 26) and are included for reproducibility.

- **A — Radio GNU / radioconda installer:**
<https://github.com/radioconda/radioconda-installer>
- **B — Pluto SDR drivers (Windows):**
<https://github.com/analogdevicesinc/plutosdr-m2k-drivers-win/releases>
- **C — Results (Google Drive folder):**
https://drive.google.com/drive/folders/1e8texmB381LSP05PVPen9b70UTNvPXuH?usp=drive_link
- **D — SU_rx.grc:**
https://drive.google.com/file/d/1ITHa5mEXnB0hbaIxwwXmWSHLIV11HDNv/view?usp=drive_link
- **E — PU_tx.grc:**
https://drive.google.com/file/d/1Isaeh9cJh6UDmF4l40ifs4dvmJa1hD4S/view?usp=drive_link
- **F — psd_logger block code:**
https://drive.google.com/file/d/1UoNh0XQouuUpJ1HS1THFbPmDtdhodbHd/view?usp=drive_link

-
- **G — Project GitHub repository:**

<https://github.com/Manish-Anwla/Generate-Datasets-using-GNU-Radio-and-SDR>

Could not be fetched automatically:

- Mid-semester “Train Logs: Link” and “Code Link” (Improvement 1/2 slides) appear as text only in `Btp1_midsem.pdf` with no embedded URL in the source files.
- Google Drive links require your Google account permissions; the report lists them but cannot verify access at compile time.

Local copies in this repository: `pluto_sdr/pluto_sdr/PU/transmitter/PU_tx.grc`, `SU/SU_rx.grc`, `SU/SU_rx_latest.py`, and measured logs under `Results/`.

Bibliography

- [1] I. F. Akyildiz *et al.*, “NeXt generation/dynamic spectrum access/cognitive radio wireless networks: a survey,” *Computer Networks*, 2006.
- [2] Collaborative learning-based spectrum sensing under partial observations, IEEE TCCN (2024) — project reference paper discussed in mid-semester documentation.